

TradePilot AI

Autonomous, explainable AI trading agents for stocks & options on Alpaca

The AI proposes. A deterministic risk engine decides. Every step is logged.

**Alpaca AI Trading Agents
Hackathon**

Paper Trading · Live

Live app: trade-pilot-ai-navy.vercel.app · API: hamzabhatti-tradepilot-ai.hf.space

AI trading bots are black boxes

- **No transparency** — you can't see why a trade was made
- **No hard risk control** — an LLM can place reckless, oversized orders
- **Especially dangerous for options** — leverage magnifies every mistake
- **Not auditable** — no record of proposal, rationale, or verdict

For real use — and for options especially — you need explainability and guardrails, not just a model that can call an order API.

Separate judgment from control

- **Multi-agent judgment** — AI produces a trade proposal WITH a plain-English rationale
- **Deterministic control** — a separate, non-AI risk engine has the final say on every order
- **Full audit trail** — proposal + risk verdict + execution result, timestamped
- **Stocks AND options** — long CALLs for bullish views, long PUTs for bearish
- **Human-in-the-loop** — optional approval queue before anything executes

AI proposes → Rules dispose → All
logged

How the pipeline fits together



Every symbol → one row in `decision_log` (proposal · risk verdict + reason · execution result) → Next.js dashboard

- **Read-only tool allowlist** — agents literally cannot see order-placement tools
- **Graceful degradation** — any failure → safe HOLD, never an uncontrolled order
- **Persistence** — SQLAlchemy → SQLite or Postgres audit log

Deterministic risk engine (pure Python)

- **Max position size** — $\leq 5\%$ of equity per trade
- **Max concentration** — $\leq 20\%$ exposure per symbol
- **Max trades / day** — hard daily cap
- **Daily-loss budget** — incl. this trade's worst-case stop-out
- **Duplicate protection** — blocks conflicting open orders
- **Required stop / target** — no naked equity orders

Why it matters

- **No AI, no network** — same inputs $\rightarrow$ same output, every time
- **pytest-covered** — the safety layer is tested
- **Overrides the AI** — the model can propose, but cannot approve
- **Every verdict logged** — with a human-readable reason

Options trading — built in, not bolted on

- **Directional view** → **option** — bullish BUY becomes a long CALL, bearish SELL a long PUT
- **Real Alpaca option chain** — near-the-money, 14–45 DTE, priced from the live ask
- **Risk-budget sizing** — number of contracts fits the position budget
- **Capped risk** — a long option's max loss is the premium paid

Verified live on the deployment:

7 real paper option orders executed — e.g. AAPL / MSFT / NVDA CALLs (bullish), TSLA PUT (bearish), each a real OCC contract with an Alpaca order id.

All three Alpaca surfaces

Trading API

REST

Account, live prices, options chain,
historical bars

MCP Server

AI data

Agents fetch data via a READ-
ONLY tool allowlist

CLI

Execution

alpaca order submit — bracket &
option orders

Most teams touch one surface. TradePilot uses all three, each for what it does best.

What it's built on

- **AI** — OpenAI Agents SDK (gpt-4o-mini)
- **Data & execution** — Alpaca Trading API + MCP server + CLI
- **Backend** — FastAPI + Uvicorn
- **Risk** — pure-Python engine (pytest)
- **Database** — SQLAlchemy + Postgres / SQLite
- **Frontend** — Next.js (App Router) + Tailwind + Recharts
- **Deploy** — Backend on Hugging Face Docker Space
- Frontend on Vercel

IT'S LIVE

Deployed & working end-to-end

- **Full scan pipeline** — watchlist → agents → risk → real paper order, in one click
- **Real executions** — stock bracket orders + option CALLs/PUTs on a live paper account
- **Explainable dashboard** — portfolio, signals, approval queue, audit timeline, backtest
- **Read-only backtester** — total return, win rate, max drawdown — never touches live path

Frontend · trade-pilot-ai-navy.vercel.app

Backend · hamzabhatti-tradepilot-ai.hf.space

Explainable. Risk-managed. Autonomous.

Stocks and options on Alpaca — where the AI proposes and the rules dispose.

- **Next** — multi-leg option strategies (spreads, hedges, overlays)
- greeks-based sizing + scheduled autonomous runs